

LAPORAN PRAKTIKUM SISTEM OPERASI

MODUL 10 SHELL



Disusun Oleh :

Ilham Lii Assidaq 2311104068

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 SOFTWARE ENGINEERING

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

1. Dasar Teori

Shell merupakan antarmuka berbasis teks pada sistem operasi Xinu yang berfungsi menerjemahkan ketikan perintah pengguna menjadi eksekusi fungsi kernel secara langsung tanpa melalui system call manual. Karakteristik utama shell Xinu bersifat internal, terintegrasi, dan statis, yang berarti seluruh perintah di dalamnya merupakan fungsi bahasa C yang menyatu dengan kernel dalam satu image sistem operasi utuh. Alur kerja shell ini beroperasi dalam sebuah loop berkelanjutan yang terdiri dari tiga tahapan utama, yaitu membaca input pengguna hingga tombol enter ditekan (Read), memecah baris input menjadi token perintah dan argumen (Parse), serta menjalankan fungsi internal yang cocok di dalam memori (Execute).

Perbedaan paling mendasar antara shell Xinu dan sistem operasi modern seperti Linux terletak pada metode penyimpanan dan eksekusi perintahnya. Di dalam sistem Linux, perintah shell merupakan program eksternal mandiri yang disimpan sebagai file executable terpisah di dalam filesystem. Sebaliknya, pada halaman dokumen dijelaskan bahwa setiap perintah di Xinu adalah fungsi internal yang sudah tertanam langsung di memori bersama kernel. Implikasinya, setiap penambahan, pengurangan, atau modifikasi terhadap perintah shell di Xinu selalu menuntut adanya perubahan kode sumber dan proses kompilasi ulang seluruh sistem dari awal.

2. Manfaat

Melalui praktikum pada modul ini, manfaat yang saya dapatkan adalah mampu memahami karakteristik unik serta alur kerja Read-Parse-Execute pada shell Xinu secara mendalam. Selain itu, saya dapat mengetahui perbedaan mendasar dalam metode eksekusi perintah antara fungsi internal Xinu yang tertanam langsung di dalam memori kernel dengan program eksternal pada sistem operasi Linux. Pengalaman praktis ini secara nyata meningkatkan keterampilan saya dalam menganalisis arsitektur shell tingkat rendah serta melatih kepekaan logika saya dalam melakukan modifikasi dan kompilasi ulang sistem secara terstruktur.

3. Jurnal

- A. [40 Poin] Akan dimodifikasi shell dengan modifikasi syscall bernama chname yang berfungsi untuk mengubah nama suatu proses. Lihat kembali modul sebelumnya cara membuat syscall. Perhatikan sekarang syscall chname mempunyai 3 parameter yaitu pid, character dan panjang character. Character untuk menyimpan nama dan panjang character untuk panjang nama.

```

Open  [icon] *prototypes.h ~xinu/include Save [icon] x
extern void arp_ntoh(struct arppacket *);
extern void arp_hton(struct arppacket *);

/* in file ascdt.c */
extern status ascdt(uint32, char *);

/* in file bufinit.c */
extern status bufinit(void);

/* in file chprio.c */
extern syscall chprio(pid32, pri16);

/* in file chname.c */
extern pri16 chname(pid32, char *, uint32);

/* in file clkupdate.S */
extern uint32 clkcount(void);

/* in file clkhandler.c */
extern interrupt clkhandler(void);

/* in file clkinit.c */
extern void clkinit(void);

/* in file clkdisp.S */
extern void clkdisp(void);

C/C++/ObjC Header Tab Width: 8 Ln 28, Col 45 INS

```

```

xinu@xinu-develop-end: ~/xinu/system
File Edit View Search Terminal Help
ctype.h ethloop.h ip.h pci.h ramdisk.h stddef.h xinu.h
date.h file.h kernel.h pipe.h rdisksys.h stdio.h
debug.h flash.h lfilesys.h ports.h resched.h stdlib.h
delay.h i386.h limits.h process.h rfilesys.h string.h
device.h icmp.h mark.h prototypes.h sdmc.h testsuite.h
xinu@xinu-develop-end:~/xinu/include$ gedit shprototypes.h
xinu@xinu-develop-end:~/xinu/include$ gedit prototypes.h
xinu@xinu-develop-end:~/xinu/include$ gedit shprototypes.h
xinu@xinu-develop-end:~/xinu/include$ cd ..
xinu@xinu-develop-end:~/xinu$ cd system
xinu@xinu-develop-end:~/xinu/system$ ls
ascdt.c DESCRIPTION getutime.c meminit.c queue.c signal.c
bufinit.c device i386.c mkbufpool.c read.c sleep.c
chname.c evic.c include net ready.c stacktrace.c
chprio.c exit.c init.c newqueue.c receive.c start.S
clkdisp.S freebuf.c initialize.c open.c recvclr.c suspend.c
clkhandler.c freemem.c insert.c panic.c recvtm.c system
clkinit.c getbuf.c insertd.c pci.c resched.c unsleep.c
close.c getc.c intr.S ptclean.c resume.c userret.c
compile getdev.c ioerr.c ptcoun.c seek.c wait.c
conf.c getitem.c ionull.c ptcree.c semcount.c wakeup.c
confiq getmem.c kill.c ptdelete.c semcreate.c write.c
control.c getpid.c kprintf.c ptinit.c semdelete.c xdone.c
COPYRIGHT getprio.c lib ptrece.c semreset.c yield.c
create.c getstk.c main.c ptrese.c send.c
ctxsw.S getticks.c mark.c ptsend.c shell
debug.c gettime.c memdemo.c putc.c signal.c
xinu@xinu-develop-end:~/xinu/system$ gedit chname.c

```

```

Open  [icon] chname.c ~xinu/system Save [icon] x
#include <xinu.h>

/*-----
 * chname - Dummy syscall
 *-----
 */
pri16 chname(
    pid32 pid,          /* ID of process to change */
    char *newname,
    uint32 namelen
)
{
    intmask mask;
    struct proctab *prptr; /* Saved interrupt mask */
                        /* Ptr to process's table entry */

    mask = disable();
    if (isbadpid(pid)) {
        restore(mask);
        return (pri16) SYSERR;
    }
    prptr = &proctab[pid];
    strncpy(prptr->prname, newname, namelen);
    prptr->prname[namelen] = '\0';

    restore(mask);
    return OK;
}

C Tab Width: 8 Ln 29, Col 2 INS

```

B. [40 Poin] Buatlah perintah baru bernama namecmd sesuai dengan langkah-langkah pada no.5 pada modul shell! Berikut adalah kode dalam perintah baru namecmd:

Berikut adalah kode dalam perintah baru namecmd:

```

printf("My new command");

// pid 3 = netin; mengubah nama proses pid 3 yaitu netin menjadi hello
chname(3,"hello",5);

return 0;

```

```
Open [icon] *xsh_namecmd.c ~xinu Save [icon] x
#include <xinu.h>
#include <stdio.h>
#include <string.h>

/* xsh_namecmd - shell command to change process name*/

shellcmd xsh_namecmd(int nargs, char *args[])
{
    printf("My new command\n");
    chname(3, "hello", 5);
    return 0;
}
```

C Tab Width: 8 Ln 17, Col 1 INS

```
xinu@xinu-develop-end:~/xinu$ ls
chname.c  COPYRIGHT  include  net      system
compile  DESCRIPTION lib      shell    xsh_mycmd.c
config   device    mutex.c  signaling.c xsh_namecmd.c
xinu@xinu-develop-end:~/xinu$ cp xsh_namecmd.c shell
xinu@xinu-develop-end:~/xinu$
```

```
xinu@xinu-develop-end:~/xinu$ ls
chname.c  COPYRIGHT  include  net      system
compile  DESCRIPTION lib      shell    xsh_mycmd.c
config   device    mutex.c  signaling.c xsh_namecmd.c
xinu@xinu-develop-end:~/xinu$ cp xsh_namecmd.c shell
xinu@xinu-develop-end:~/xinu$ cd shell
xinu@xinu-develop-end:~/xinu/shell$ ls
addargs.c  xsh_cat.c      xsh_help.c      xsh_namecmd.c  xsh_sleep.c
lexan.c    xsh_clear.c    xsh_kill.c       xsh_netinfo.c  xsh_tee.c
Notes_On_Pipes xsh_date.c     xsh_ls.c        xsh_ns.c       xsh_udpdump.c
shell.c    xsh_devdump.c  xsh_memdump.c    xsh_ping.c     xsh_udpecho.c
xsh_argecho.c xsh_echo.c     xsh_memstat.c    xsh_ps.c       xsh_udpserver.c
xsh_arp.c   xsh_exit.c     xsh_mycmd.c      xsh_rdstest.c  xsh_uptime.c
xinu@xinu-develop-end:~/xinu/shell$ gedit xsh_namecmd.c
xinu@xinu-develop-end:~/xinu/shell$
```

```
Open [icon] *shprototypes.h ~xinu/include Save [icon] x
extern shellcmd xsh_ping (int32, char *[]);

/* in file xsh_ps.c */
extern shellcmd xsh_ps (int32, char *[]);

/* in file xsh_sleep.c */
extern shellcmd xsh_sleep (int32, char *[]);

/* in file xsh_udpdump.c */
extern shellcmd xsh_udpdump (int32, char *[]);

/* in file xsh_udpecho.c */
extern shellcmd xsh_udpecho (int32, char *[]);

/* in file xsh_udpserver.c */
extern shellcmd xsh_udpserver (int32, char *[]);

/* in file xsh_uptime.c */
extern shellcmd xsh_uptime (int32, char *[]);

/* in file xsh_mycmd.c */
extern shellcmd xsh_mycmd (int32, char *[]);

/* in file xsh_namecmd.c */
extern shellcmd xsh_namecmd (int32, char *[]);

/* in file xsh_help.c */
extern shellcmd xsh_help (int32, char *[]);

C/C++/ObjC Header Tab Width: 8 Ln 82, Col 1 INS
```

```

File Edit View Search Terminal Help
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"" -I../i
hclude -o binaries/xsh udpserver.o ../shell/xsh udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"" -I../i
hclude -o binaries/xsh uptime.o ../shell/xsh uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"" -I../i
hclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"" -I../i
hclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"" -I../i
hclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"" -I../i
hclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"" -I../i
hclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$ sudo minicom
[sudo] password for xinu:

```

C. [20 Poin] Test hasilnya:

- Masuk ke terminal xinu
- Jalankan perintah ps

```

Welcome to Xinu!

xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr  Stack Size
-----
0 prnull          ready  0    0  0x005FDFFC 0x00FFFD8  8192
1 ipout           wait   500  0  0x005FBFFC 0x005FBF30  8192
2 netin           wait   500  0  0x005F9FFC 0x005F9ED0  8192
4 Main process    recv   20   3  0x005E7FFC 0x005E7F44 65536
5 shell           recv   50   4  0x005D7FFC 0x005D79AC  8192
6 ps              curr   20   5  0x005F7FFC 0x005F7FC4  8192

xsh $

```

c. Jalankan perintah namecmd

```

xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr  Stack Size
-----
0 prnull          ready  0    0  0x005FDFFC 0x00FFFD8  8192
1 ipout           wait   500  0  0x005FBFFC 0x005FBF30  8192
2 netin           wait   500  0  0x005F9FFC 0x005F9ED0  8192
4 Main process    recv   20   3  0x005E7FFC 0x005E7F44 65536
5 shell           recv   50   4  0x005D7FFC 0x005D79AC  8192
6 ps              curr   20   5  0x005F7FFC 0x005F7FC4  8192

xsh $ namecmd
My new command
xsh $

```

d. Jalankan perintah ps

```

xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr  Stack Size
-----
0 prnull          ready  0    0  0x005FDFFC 0x00FFFD8  8192
1 ipout           wait   500  0  0x005FBFFC 0x005FBF30  8192
2 netin           wait   500  0  0x005F9FFC 0x005F9ED0  8192
4 Main process    recv   20   3  0x005E7FFC 0x005E7F44 65536
5 shell           recv   50   4  0x005D7FFC 0x005D79AC  8192
6 ps              curr   20   5  0x005F7FFC 0x005F7FC4  8192

xsh $ namecmd
My new command
xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr  Stack Size
-----
0 prnull          ready  0    0  0x005FDFFC 0x00FFFD8  8192
1 ipout           wait   500  0  0x005FBFFC 0x005FBF30  8192
2 netin           wait   500  0  0x005F9FFC 0x005F9ED0  8192
4 Main process    recv   20   3  0x005E7FFC 0x005E7F44 65536
5 shell           recv   50   4  0x005D7FFC 0x005D79AC  8192
8 ps              curr   20   5  0x005F7FFC 0x005F7FC4  8192

xsh $

```

4. Kesimpulan

saya dapat menyimpulkan bahwa saya telah berhasil memahami karakteristik unik serta arsitektur internal dari shell sistem operasi Xinu. Saya kini mengerti bagaimana alor kerja Read, Parse, dan Execute beroperasi secara dinamis dalam sebuah loop berkelanjutan untuk memproses setiap baris input teks yang saya masukkan hingga menjadi eksekusi fungsi kernel yang sesuai.

Selain itu, saya juga berhasil menganalisis perbedaan struktural yang mendasar antara mekanisme eksekusi perintah pada Xinu dan Linux. Melalui pemahaman ini, saya mengetahui bahwa jika Linux mengandalkan program eksternal yang berdiri sendiri di dalam filesystem, Xinu justru mengintegrasikan seluruh perintahnya sebagai fungsi C internal yang tertanam langsung di dalam memori bersama kernel. Pengalaman praktis ini tidak hanya memperluas wawasan teoretis saya mengenai sistem operasi terdistribusi, tetapi juga melatih keterampilan saya dalam memahami bagaimana modifikasi kode sumber dan kompilasi ulang sistem dilakukan untuk membangun sebuah image OS yang utuh.